Abilità Informatiche 2026/27

L02 · Come si rappresenta l'informazione

Andrea Schimmenti · Università di Bologna

Di che cosa parliamo oggi

1. Il **bit**: l'unità minima
2. Contare in **binario** (e in **esadecimale**)
3. Dai numeri alle **lettere**: ASCII, Unicode, UTF-8
4. **Caratteri, glifi** e direzione della scrittura
5. **Immagini, suoni, video**, compressione, metadati
6. Dove stanno i dati
7. **File e formati**

1. Il bit

Due stati

- **Bit** = *binary digit*, cifra binaria
- La più piccola quantità di informazione: **0 o 1**
- Fisicamente: corrente sì/no, particella magnetica orientata in un verso o nell'altro, avvallamento su un CD

Perché **due** stati? Distinguere "c'è corrente" da "non c'è" è molto più **affidabile** che distinguere dieci livelli di tensione.

Ogni bit in più raddoppia le possibilità

Bit	Combinazioni	Che cosa ci sta
1	2	vero / falso
2	4	le quattro stagioni
3	8	i giorni della settimana (+1)
7	128	l'alfabeto inglese, cifre, punteggiatura (**ASCII**)
8	256	un **byte**
16	65.536	gran parte degli ideogrammi cinesi comuni
n	2^n	

Byte e multipli

Unità	Valore	Ordine di grandezza
kilobyte (kB)	mille byte	una pagina di testo semplice
megabyte (MB)	un milione	un romanzo in testo, una foto compressa
gigabyte (GB)	un miliardo	un film in alta definizione
terabyte (TB)	mille miliardi	il disco di un computer recente
petabyte (PB)	un milione di miliardi	una grande biblioteca digitale

1000 o 1024? kB = 1000 byte, KiB = 1024 byte. Per questo un disco "da 1 TB" appare come circa 931 GB.

2. Contare in binario

Il sistema posizionale

In **base 10**: 1819 = 1 × 1000 + 8 × 100 + 1 × 10 + 9 × 1

In **base 2** ogni posizione vale il **doppio** della precedente:

128	64	32	16	8	4	2	1	
0	1	0	0	0	0	0	1	= 64 + 1 = **65**
0	0	0	1	0	0	1	1	= 16 + 2 + 1 = **19**

Ricordate il **65**: tra poco diventa una lettera.

Perché l'esadecimale

- **Base 16**: cifre 0–9 e lettere A–F
- **Una cifra esadecimale = 4 bit** → un byte si scrive con **due cifre**

Binario	`0100 0001`	`1100 0011`	`1111 1111`
Esadecimale	`41`	`C3`	`FF`
Decimale	65	195	255

Lo vedete ovunque: colori del web (`#980260`), codici Unicode (`U+00E8`).

Esercizio (5 minuti)

1. Quanto vale `00001010`? E `00011001`?
2. Come si scrive 7 in binario? E 100?
3. Il colore `#FF0000` è fatto di tre byte (rosso, verde, blu). Che colore è?

3. Dai numeri alle lettere

Una tabella condivisa

- Il computer **non conosce le lettere**: conosce solo numeri
- Serve una **codifica dei caratteri**: una tabella che associa a ogni carattere un numero
- Chi scrive e chi legge devono usare **la stessa tabella**

ASCII (1963): un alfabeto per una sola lingua

- 7 bit → **128 caratteri**
- caratteri di controllo, spazio, cifre, punteggiatura, 26 lettere inglesi
- `A` = 65 · `B` = 66 · `a` = 97 · spazio = 32
- `Ciao` → 67 105 97 111

Che cosa manca? è, ñ, ç, ß, ø… e ogni alfabeto non latino.

Il caos delle codifiche nazionali

L'ottavo bit aggiunge altri 128 caratteri, ma **ognuno a modo suo**:

- **Latin-1** (ISO 8859-1), **Windows-1252**: Europa occidentale
- **ISO 8859-2**: Europa centrale · **ISO 8859-5**, **KOI8-R**: cirillico
- **Shift-JIS**: giapponese · **GB 2312**: cinese semplificato · **Big5**: tradizionale

Il numero **232**: `è` in Latin-1 · `č` in ISO 8859-2 · `Ш` in ISO 8859-5

Il *mojibake* (□□□□)

`è` in UTF-8 = due byte: `C3 A8`

Letti come se fossero Latin-1: `C3` → `Ã` · `A8` → `¨`

Scritto	Letto con la tabella sbagliata
perché	perchÃ©
città	cittÃ

I byte sono giusti. È sbagliata la tabella.

Il caso opposto produce il carattere sostitutivo `�`.

Unicode (1991): un numero per ogni carattere

Carattere	Punto di codice	Nome
A	U+0041	LATIN CAPITAL LETTER A
è	U+00E8	LATIN SMALL LETTER E WITH GRAVE
ж	U+0436	CYRILLIC SMALL LETTER ZHE
ع	U+0639	ARABIC LETTER AIN
𐤀	U+4E2D	CJK UNIFIED IDEOGRAPH-4E2D
😀	U+1F600	GRINNING FACE

Oltre **150.000 caratteri**, più di **160 scritture**: dall'arabo al cuneiforme, dalla lineare B alle rune.

UTF-8 (1992): come si scrive Unicode in byte

Carattere	Punto di codice	Byte in UTF-8
`A`	U+0041	`41` (1 byte)
`è`	U+00E8	`C3 A8` (2)
`Ж`	U+0436	`D0 B6` (2)
`中`	U+4E2D	`E4 B8 AD` (3)
😀	U+1F600	`F0 9F 98 80` (4)

- **Compatibile con ASCII**: un testo ASCII è già UTF-8
- **Universale**: italiano, russo, arabo e giapponese nello stesso file
- Lettere ≠ byte: un testo russo occupa circa il doppio di uno inglese

Una lettera, due codifiche

`é` può essere:

- **un carattere**: U+00E9
- **due caratteri**: `e` (U+0065) + accento combinante (U+0301)

A schermo identici, per il computer **diversi**.

Cercate *café* in un corpus e trovate solo **metà** delle occorrenze?
→ serve la **normalizzazione** (NFC / NFD), il primo passo di ogni analisi linguistica.

Le emoji sono caratteri

- Ogni emoji ha un **punto di codice** e un **nome**
- Ogni sistema le **disegna** a modo suo
- Alcune sono **sequenze**:
  - 👍🏽 = pollice + modificatore di carnagione
  - 👩‍💻 = donna + *zero width joiner* + computer
  - 🇮🇹 = lettere regionali I + T

4. Caratteri e glifi

Carattere e glifo

- **Carattere**: l'unità astratta ("la *a* minuscola", U+0061)
- **Glifo**: la forma concreta con cui viene disegnato
- Il **file** contiene i caratteri, il **font** fornisce i glifi

In arabo ogni lettera ha forme diverse se isolata, iniziale, mediana o finale: il file contiene **un solo carattere**, il sistema sceglie la forma.

Il *tofu* □

Il carattere è giusto, ma **manca il font** per disegnarlo.

Mojibake (Ã¨, Ð¶)

→ problema di **codifica**

→ indicare la tabella giusta

Tofu (□)

→ problema di **font**

→ installare un font adatto

Google Noto: "*no more tofu*".

Destra, sinistra e chi decide

- Arabo, ebraico, persiano, urdu: da **destra a sinistra**, ma i numeri da sinistra a destra → **algoritmo bidirezionale**
- Chi decide quali caratteri entrano in Unicode? Il **Consorzio Unicode**, i cui membri con pieno voto sono soprattutto **grandi aziende**
- Molte scritture di comunità minoritarie sono arrivate tardi o **mancano ancora**
- **Unificazione Han**: ideogrammi cinesi, giapponesi e coreani "uniti" sotto lo stesso punto di codice

5. Immagini, suoni, video

Le immagini

- *Bitmap*: griglia di **pixel**
- **Risoluzione**: 12 megapixel = 4000 × 3000 pixel
- **Profondità di colore**: RGB, un byte per canale → oltre 16 milioni di colori
- 12 milioni × 3 byte ≈ **36 MB** non compressa · 3–5 MB in JPEG
- **Vettoriali** (SVG, font, loghi): forme geometriche, ingrandibili senza perdere nitidezza

Suoni e video

Suono: si misura l'onda molte volte al secondo (**campionamento**)

44.100 campioni × 2 byte × 2 canali × 60 s ≈ **10,6 MB** al minuto (CD)

Lo stesso minuto in MP3: circa **1 MB**

Video: 24–30 immagini al secondo + audio → senza compressione, **gigabyte** al minuto

Compressione

Senza perdita (*lossless*)	Con perdita (*lossy*)
ZIP, PNG, FLAC	JPEG, MP3, MP4
sfrutta ripetizioni e regolarità	elimina ciò che si percepisce meno
l'originale si ricostruisce **bit per bit**	l'originale **non si recupera**
per **conservare**	per **diffondere**

I metadati nascosti

- Le foto contengono dati **EXIF**: data, ora, modello del telefono e spesso le **coordinate GPS**
- Un documento Word conserva il **nome dell'autore** e parte delle modifiche

Pubblicare una foto scattata a casa può rivelare **dove abitate**.

Dove stanno i dati

- Mentre un programma li usa: nella **RAM**
- Quando li salviamo: nella **memoria di massa** (disco, SSD)
- Quando li salviamo "nel *cloud*": sui dischi dei **server** di un'azienda, in un **data center** con una sede, un proprietario e una legge di riferimento *(L06)*

6. File e formati

File, formato, estensione

- **File**: sequenza di byte con un nome
- **Formato**: la convenzione che dice come interpretare quei byte
- **Estensione** (`.txt`, `.docx`, `.jpg`): solo un'indicazione per il sistema operativo

Rinominare `foto.jpg` in `foto.txt` **non** trasforma l'immagine in testo.

Tre grandi famiglie

Tipo	Esempi	Contenuto
Testo semplice	`.txt` `.csv` `.md` `.html` `.xml` `.json`	solo caratteri
Documenti formattati	`.docx` `.odt` `.pdf` `.epub`	testo + impaginazione + immagini
Multimediali	`.png` `.jpg` `.mp3` `.mp4`	immagini, suoni, video

Prova: rinominate una copia di un `.docx` in `.zip` e apritela. Il testo è in `word/document.xml`.

Il PDF

- Progettato per apparire **identico ovunque**
- Descrive la **posizione** di ogni carattere, più che la struttura del testo
- Copiando: parole spezzate, righe interrotte, note mescolate al testo, colonne nell'ordine sbagliato
- Un PDF da scansione può contenere **solo immagini**, senza testo

Formati aperti e conservazione

- **Aperti**: specifiche pubbliche, chiunque può leggerli (testo, HTML, XML, PNG, ODT, PDF)
- **Proprietari**: controllati da un'azienda
- WordStar, vecchio WordPerfect, floppy disk: oggi **difficili da leggere**
- Il rischio di una *digital dark age*

Per un testo che deve durare: **formato aperto e semplice**.

Per l'analisi linguistica: **testo semplice in UTF-8**.

Attività

1. **Il vostro nome in binario**: scrivete l'iniziale in binario (ASCII) e incollatela in chat
2. **Caccia al mojibake**: aprite i due file di prova (UTF-8 e Latin-1) nel browser e in un editor
3. **Esplorare Unicode**: cercate su *codepoints.net* un carattere della lingua che studiate. Punto di codice? Blocco? Quanti byte in UTF-8?
4. **Dentro un .docx**: rinominatelo in `.zip` e aprite `word/document.xml`

Riassumendo

- Tutto è fatto di **bit**; 8 bit = un **byte**
- Il **binario** e l'**esadecimale** sono solo modi di scrivere numeri
- Le lettere sono numeri secondo una **tabella**: ASCII → codifiche nazionali → **Unicode** e **UTF-8**
- **Mojibake** = tabella sbagliata · **tofu** = font mancante · **normalizzazione** = stessa lettera, stessa codifica
- Immagini e suoni si **campionano** e si **comprimono**; i file nascondono **metadati**
- **Formato** ≠ estensione; per durare e per analizzare: **testo semplice UTF-8**

Prossima lezione

L03 · Strutturare i dati

Liste, tabelle, alberi, grafi, marcatura e grafi di conoscenza